```
    var window: UIWindow?

    private let navigator = Navigator(configuration: .init(
        name: "main",
        startLocation: baseURL.appending(path: "hikes")
    ))

»   func scene(
»       _ scene: UIScene,
»       willConnectTo session: UISceneSession,
»       options connectionOptions: UIScene.ConnectionOptions
»   ) {
    }
}
```

The function being called is our trigger to start rendering some content.

Functions in Swift

Functions in Swift look pretty different than methods in Ruby. Like properties of a class, each parameter in Swift must also have a type.

The following function takes two parameters of type `Int` and returns another `Int`:

```
func add(x: Int, y: Int) -> Int {
    x + y
}

add(x: 1, y: 2) // Returns 3
```

We call functions in Swift just like we call Ruby methods, with named parameters. We can *omit* a parameter from the call site by using an underscore:

```
func add(_ x: Int, _ y: Int) -> Int {
    x + y
}

add(1, 2) // Returns 3
```

Or we can *rename* a parameter for the implementation:

```
func add(first x: Int, second y: Int) -> Int {
    x + y
```